

UGF AgentX — Backend Implementation Status

Scope	Backend only (/Backend)
Reference	UGF_AgentX_Hackathon_Plan_260518_120803.pdf
Last updated	May 19, 2026
Source of truth	Backend/src/ + server.ts route mounting

Table of Contents

1. [Executive Summary](#)
2. [Status Legend](#)
3. [Architecture](#)
4. [Tech Stack](#)
5. [API Endpoints](#)
6. [Implemented \(Detail\)](#)
7. [Partially Implemented](#)
8. [Remaining Work \(Detail\)](#)
9. [Database Schema](#)
10. [Environment Variables](#)
11. [Frontend ↔ Backend Integration](#)
12. [Judge Demo Checklist](#)
13. [File Inventory](#)
14. [Quick Start](#)

Executive Summary

Area	Progress	Notes
Server & middleware	100%	Express, CORS, logging, global errors
Wallet auth (JWT)	95%	Nonce + signature + Bearer on protected routes
Chat & intent	95%	Rule parser + Gemini fallback + persistence
Chat sessions API	100%	List, history, delete, auto-title on first message
Timeline steps in chat	100%	aiSteps + transactionSteps via responseEngine.ts
Transaction / gallery / wallet APIs	90%	CRUD + reads; some routes lack auth
UGF on-chain execution	75%	Live in src/ ; needs env, contract, E2E demo
Smart contract in repo	0%	No Solidity / deployed ABI checked in
Production hardening	40%	No Zod, partial env validation

Overall backend readiness: ~78% for hackathon MVP (chat + auth + DB strong; on-chain path code-complete but ops-dependent).

What works today

- Wallet login: `POST /api/auth/nonce` → sign → `POST /api/auth/verify` → JWT
- Authenticated chat: intent, reply, gas estimate (mock formula), Base64 `tokenURI` , timeline steps
- Session sidebar: `GET /api/chat/sessions` , `GET /api/chat/history/:id` , `DELETE /api/chat/sessions/:id`
- Transaction history, NFT gallery, wallet summary (DB-backed)
- `POST /api/ugf/execute` — full quote → settle → execute → poll (when env + contract configured)

What still blocks a polished judge demo

- ERC721 contract source + verified `NFT_CONTRACT_ADDRESS` on Base Sepolia
- UGF credentials (`UGF_API_KEY` , signer key matching user wallet or demo wallet funded)
- Chat gas estimate still **formula-based**, not live UGF `quote.get()` preview
- Updating the **same** `transactions` row from chat → UGF execute (today chat creates `pending` , UGF may insert another row)
- On-chain ETH / Mock USD balance sync to `users` table
- `analytics` table unused; `intentParser.ts` legacy file unused

Status Legend

Icon	Status	Meaning
✓	Done	Implemented in active <code>src/</code> and mounted
⚠	Partial	Works with limits, mock data, or not fully wired E2E
✗	Missing	Not implemented or not used
🔒	Auth	Requires Authorization: Bearer <jwt>

Architecture

System context

UGF AgentX uses a **standalone Express API** (port 5000), not Next.js API routes. The Vite/React frontend calls this API with a JWT after wallet signature login. Supabase (Postgres) is the system of record for users, chats, transactions, and badges. On-chain actions go through **UGF testnet SDK** (`@tychilabs/ugf-testnet-js`) on **Base Sepolia**, targeting an ERC721-style contract via `mintBadge` / `donate` calldata.

High-level diagram

ARCHITECTURE DIAGRAM (MERMAID SOURCE)

```

flowchart TB
    subgraph Client["Frontend (Vite + React)"]
        UI[Chat / Sidebar / Wallet Panel]
        W3[Wagmi + ConnectKit]
        Auth[WalletAuthSync – sign nonce]
    end

    subgraph API["Backend – Express :5000"]
        S[server.ts]
        H[health.ts]
        A[auth.ts]
        C[chat.ts]
        T[transaction.ts]
        U[ugf.ts]
        AM[authMiddleware]
        S --> H
        S --> A
        S --> C
        S --> T
        S --> U
        C --> AM
        T --> AM
        U --> AM
    end

    subgraph Services["Services layer"]
        RE[responseEngine.ts]
        UGF[ugfService.ts]
        US[userService.ts]
        NS[nonceStore.ts]
        C --> RE
        U --> UGF
        A --> US
        A --> NS
    end

    subgraph External["External"]
        GEM[Gemini API]
        SB[(Supabase)]
        UGFAPI[UGF Cloud API]
        RPC[Base Sepolia RPC]
        SC[NFT Contract]
    end

    UI --> Auth
    Auth --> A
    UI -->|Bearer JWT| C
    UI -->|Bearer JWT| T
    UI -->|Bearer JWT| U
    C --> GEM
    C --> SB
    T --> SB
    U --> UGF
    UGF --> UGFAPI
    UGF --> RPC
    UGF --> SC

```

Layer responsibilities

Layer	Files	Responsibility
HTTP	server.ts , routes/*	Routing, request validation (minimal), JSON responses
Auth	auth.ts , authMiddleware.ts , nonceStore.ts	Wallet nonce, viem verifyMessage , JWT issue/verify
Chat	routes/chat.ts	Intent parse, Gemini fallback, session/message CRUD, metadata, mock gas
Timeline	services/responseEngine.ts	Per-intent aiSteps + transactionSteps templates
Chain	routes/ugf.ts , services/ugfService.ts	Calldata build, UGF quote/settle/execute, status poll
Data	config/supabase.ts , services/userService.ts	Supabase admin client, user upsert
Cross-cutting	middleware/cors.ts , errorHandler.ts , logger.ts	CORS, AppError , request logs

Wallet authentication flow

ARCHITECTURE DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    participant FE as Frontend
    participant Auth as /api/auth/*
    participant DB as Supabase

    FE->>Auth: POST /auth/nonce { walletAddress }
    Auth-->>FE: nonce message (UUID-based)
    FE->>FE: wallet.signMessage(nonce)
    FE->>Auth: POST /auth/verify { walletAddress, signature }
    Auth->>Auth: verifyMessage (viem)
    Auth->>DB: ensureUser(wallet)
    Auth-->>FE: JWT (7d) + user profile
    Note over FE: apiFetch adds Authorization Bearer
```

Chat message flow (implemented)

ARCHITECTURE DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    participant FE as Frontend
    participant Chat as POST /api/chat
    participant RE as responseEngine
    participant DB as Supabase
    participant AI as Gemini

    FE->>Chat: Bearer JWT + { sessionId?, message }
    Chat->>Chat: ensureSession (title from 1st message)
    Chat->>DB: insert user message
    Chat->>Chat: ruleBasedParse()
    alt intent === UNKNOWN
        Chat->>AI: JSON classify (model chain + retries)
        AI-->>Chat: intent, recipient, amount, reply
    end
    Chat->>RE: getStepsForIntent(intent)
    Chat->>Chat: buildTokenURI, buildGasEstimate (mock)
    Chat->>DB: ai_actions, transactions (pending), minted_badges?
    Chat->>DB: insert assistant message
    Chat-->>FE: reply, intent, tokenURI, gasEstimate, aiSteps, transactionSteps, sessionId, transactionId
```

Chat sessions sidebar flow (implemented)

ARCHITECTURE DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    participant FE as Frontend
    participant API as /api/chat/sessions
    participant Hist as /api/chat/history/:id
    participant DB as Supabase

    FE->>API: GET /chat/sessions?walletAddress=
    API->>DB: chat_sessions (max 50, by updated_at)
    API->>DB: message counts + title fallback from first user msg
    API-->>FE: sessions[]

    FE->>Hist: GET /chat/history/:sessionId
    API->>DB: assertSessionOwner
    Hist-->>FE: messages[]

    FE->>API: DELETE /chat/sessions/:id
    API->>DB: delete messages + session
```

UGF execution flow (implemented in code; ops-dependent)

ARCHITECTURE DIAGRAM (MERMAID SOURCE)

```
sequenceDiagram
    participant FE as Frontend
    participant UGF as POST /api/ugf/execute
    participant SVC as ugfService
    participant SDK as UGF SDK
    participant DB as Supabase
    participant Chain as Base Sepolia

    FE->>UGF: Bearer JWT + intent, userWallet, tokenURI, sessionId, ...
    UGF->>UGF: encodeFunctionData (mintBadge | donate)
    UGF->>SVC: executeUgfFlow({ from, to, data })
    SVC->>SDK: auth.login(signer)
    SVC->>SDK: quote.get (TYI Mock USD)
    SVC->>SDK: vault.payAndSubmit | x402.execute
    SVC->>SDK: chains.evm.sponsorAndExecute
    SVC->>SDK: poll status (30s)
    SDK->>Chain: sponsored tx
    Chain-->>SDK: txHash
    UGF->>DB: insert transactions + minted_badges (on success)
    UGF-->>FE: success + txHash | failure + step
```

PDF vs actual architecture

Aspect	Hackathon PDF	Actual backend	Match
API host	Next.js API routes	Express 4 @ :5000	⚠️ Different host, same REST shape
Auth	Wallet connect	Nonce + signature + JWT	✅ Stronger than PDF
Intent	Rule-based	Rule + Gemini JSON fallback	✅
AI provider	OpenAI	Google Gemini	⚠️ Equivalent role
Database	Supabase	Supabase service role	✅
Gasless tx	UGF SDK	ugfService.ts + route mounted	✅ Code ready
Contract	Solidity ERC721	External address only	❌ Not in repo

Tech Stack

Layer	Technology	Status	Used in
Runtime	Node.js (ESM)	✓	<code>server.ts</code>
Framework	Express 4	✓	All routes
Language	TypeScript ~5.8	✓	<code>src/</code>
Database	Supabase (Postgres)	✓	All persistence
AI	<code>@google/generative-ai</code>	✓	<code>chat.ts</code>
Auth	<code>jsonwebtoken</code> , <code>viem</code>	✓	<code>auth.ts</code> , middleware
Blockchain	<code>@tychilabs/ugf-testnet-js</code>	✓	<code>ugfService.ts</code>
Blockchain	<code>viem</code>	✓	Auth verify, UGF calldata
Blockchain	<code>ethers</code> Wallet	✓	UGF signer in <code>ugfService.ts</code>
Validation	Zod	✗	In package.json, unused
Legacy	<code>openai</code>	✗	Not imported

API Endpoints

Public

Method	Endpoint	Description
GET	<code>/health</code>	Liveness { status: "ok" }
POST	<code>/api/auth/nonce</code>	Issue sign-in message for wallet
POST	<code>/api/auth/verify</code>	Verify signature, return JWT + user

Protected (🔒 Authorization: Bearer <token>)

Method	Endpoint	Description
POST	/api/chat	Parse message, persist, return reply + steps + metadata
GET	/api/chat/sessions? walletAddress=	List sessions (50 max), titles, message counts
DELETE	/api/chat/sessions/:sessionId	Delete session + all messages (owner only)
GET	/api/chat/history/:sessionId	Messages oldest → newest (owner only)
POST	/api/transaction	Create transaction row manually
GET	/api/transactions/:walletAddress	Transaction history for wallet
GET	/api/gallery/:walletAddress	Minted badges / NFT metadata
GET	/api/wallet?walletAddress=	User row (balances from DB)
POST	/api/ugf/execute	UGF quote → settle → execute on-chain

Unauthenticated (gap)

Method	Endpoint	Issue
GET	/api/transaction/history/:sessionId	No JWT; only checks session exists

Implemented (Detail)

1. Server foundation — ✅ 100%

- Express bootstrap, JSON body parser, URL-encoded support
- CORS from `FRONTEND_URL` (comma-separated origins)
- Per-request logging (`METHOD` `path`)
- Global `errorHandler` with `AppError` (400/403/500)
- 404 JSON for unknown routes
- `validateConfig()` warns on missing Gemini / Supabase keys (non-fatal)

2. Wallet authentication — ✅ 95%

Files: `routes/auth.ts` , `middleware/authMiddleware.ts` , `services/nonceStore.ts` , `services/userService.ts`

Feature	Detail
Nonce	<code>POST /api/auth/nonce</code> — validates address (via <code>isAddress</code>), stores in-memory nonce
Verify	<code>POST /api/auth/verify</code> — <code>verifyMessage</code> , creates/loads user in <code>users</code>
JWT	7-day token: <code>{ walletAddress, userId }</code> signed with <code>JWT_SECRET</code>
Middleware	<code>authMiddleware</code> on chat, transaction, ugf routes
Wallet match	<code>assertWalletAccess</code> — query/path wallet must match JWT address

Not done: refresh tokens, nonce persistence across server restarts (in-memory only), rate limiting.

3. Chat API — 95%

File: `routes/chat.ts` (~890 lines)

Intents

Intent	Triggers	Action type in DB
<code>MINT_BADGE</code>	mint, badge	<code>mint_badge</code>
<code>CLAIM_CERT</code>	claim, certificate, cert	<code>claim_cert</code>
<code>DONATE</code>	donate, donation, contribute	<code>donate</code>
<code>SEND_REWARD</code>	reward, tip, bonus	<code>send_reward</code>
<code>UNKNOWN</code>	—	Gemini fallback or help reply

Entity extraction

- ETH address: `0x` + 40 hex
- Name recipient: `for|to|recipient <name>`
- Amount: number before USD / USDC / Mock USD

Session lifecycle

- `ensureSession()` — creates UUID session or reuses valid `sessionId`
- **Auto-title:** first message → `deriveSessionTitle()` (40 chars + ...)
- Upsert updates `updated_at` on each message
- Owner checks via `assertSessionOwner()` on history/delete

`POST /api/chat` **response fields**

Field	Source
reply	Rule template or Gemini
intent , recipient , amount , confidence	Parser
tokenURI	Base64 JSON + SVG (buildTokenURI) for mint/claim
gasEstimate	Mock formula (buildGasEstimate) — not UGF quote API
aiSteps , transactionSteps	getStepsForIntent()
sessionId , transactionId	DB inserts

Persistence pipeline

1. Ensure session (+ title if new)
2. Insert chat_messages (user)
3. Parse intent (rules → Gemini if UNKNOWN)
4. Insert ai_actions
5. Insert transactions (status pending)
6. Insert minted_badges if MINT_BADGE
7. Insert chat_messages (assistant)

4. Chat sessions API — 100%

Endpoint	Behavior
GET /api/chat/sessions	Max 50 sessions, updated_at DESC , messageCount , title fallback from first user message
GET /api/chat/history/:sessionId	Full thread, ownership enforced
DELETE /api/chat/sessions/:sessionId	Cascading delete messages then session

5. Transaction timeline — 100%

File: services/responseEngine.ts

- Per-intent flows: MINT_BADGE , CLAIM_CERT , DONATE , SEND_REWARD , UNKNOWN
- aiSteps : timed assistant narration for chat UI
- transactionSteps : UGF pipeline labels (quote, settle, execute, confirm, save)
- Returned on every successful POST /api/chat (non-UNKNOWN gets full tx steps)

6. Gemini AI fallback — ~90%

- JSON mode + brace-extraction fallback
- Model chain: GEMINI_MODEL env → gemini-2.0-flash → 1.5-flash-latest → 1.5-pro
- Retries on 429/5xx (GEMINI_MAX_RETRIES , GEMINI_RETRY_DELAY_MS)
- Skips cleanly if no API key
- Confidence: rule-based | GeminiAI | failed

7. NFT metadata (off-chain) — 90%

- SVG badge/certificate in `buildSvg()`
- `data:application/json;base64,...` token URI
- Traits: Type, Recipient, Issued By, Network (Base Sepolia)

8. Transaction, gallery, wallet APIs — 90%

Endpoint	Purpose
<code>POST /api/transaction</code>	Manual tx row create
<code>GET /api/transactions/:walletAddress</code>	Wallet tx history
<code>GET /api/gallery/:walletAddress</code>	<code>minted_badges</code> list
<code>GET /api/wallet</code>	User profile + DB balances

9. UGF on-chain execution — 75% (code complete)

Files: `routes/ugf.ts` , `services/ugfService.ts`

Step	Implementation
Quote	<code>client.quote.get</code> with TYI Mock USD on Base Sepolia
Settle	<code>vault.payAndSubmit</code> or <code>x402.execute</code> by payment mode
Execute	<code>client.chains.evm.sponsorAndExecute</code>
Confirm	Poll <code>client.status.get</code> up to 30s
Calldata	<code>mintBadge(to, tokenURI)</code> or <code>donate(to, amount)</code> via viem
DB	Inserts <code>transactions</code> + <code>minted_badges</code> on success; partial failure JSON on step errors

Requires env: `UGF_API_KEY` , `UGF_SIGNER_PRIVATE_KEY` (must match `userWallet` or demo fails), `NFT_CONTRACT_ADDRESS` , `BASE_SEPOLIA_RPC_URL` .

10. Supabase schema usage — ~90%

Table	Used
<code>users</code>	 Auth, wallet API
<code>chat_sessions</code>	
<code>chat_messages</code>	
<code>transactions</code>	 Chat + UGF
<code>minted_badges</code>	 Chat + UGF
<code>ai_actions</code>	 Audit log
<code>analytics</code>	 Never written

Partially Implemented

Feature	What exists	What's missing
UGF end-to-end demo	Full <code>executeUgfFlow</code> in <code>src/</code>	Contract deploy, API keys, signer = user wallet, frontend calls on confirm
Gas estimate in chat	Mock USD formula	Real <code>quote.get()</code> preview + store <code>ugf_quote_id</code> on chat tx
Transaction lifecycle	Chat creates <code>pending</code> row	<code>PATCH</code> to update same row after UGF; link <code>transactionId</code> in execute body
Wallet balances	Columns on <code>users</code>	No RPC sync; UI shows DB defaults
UGF execute vs chat tx	Both insert transactions	Should update chat's <code>transactionId</code> instead of duplicate rows
Session tx history route	<code>GET /api/transaction/history/:sessionId</code>	No auth middleware
Env documentation	<code>.env.example</code> partial	UGF + contract vars not in example file
Request validation	Ad-hoc checks	No Zod schemas
Legacy <code>intentParser.ts</code>	Old swap/history intents	Unused; should delete or merge

Remaining Work (Detail)

Critical — blocks live on-chain demo

ID	Task	Detail	Effort
C1	Deploy ERC721 contract	<code>mintBadge(address,string)</code> (+ <code>donate</code> if needed) on Base Sepolia	High
C2	Document / add contract to repo	<code>contracts/</code> , ABI JSON, address in <code>.env</code>	Medium
C3	UGF environment setup	<code>UGF_API_KEY</code> , <code>UGF_SIGNER_PRIVATE_KEY</code> , fund Mock USD vault	Low
C4	E2E UGF test	curl or frontend → <code>POST /api/ugf/execute</code> → real <code>tx_hash</code>	Medium
C5	Signer = user wallet	Today backend signs with server key; must match connected wallet or use user signing flow	High

High — demo quality & PDF alignment

ID	Task	Detail	Effort
H1	UGF gas quote in chat	Call <code>quote.get()</code> for preview; fallback to mock; save <code>ugf_quote_id</code>	Medium
H2	Update transaction on execute	Accept <code>transactionId</code> from chat in <code>ugf/execute</code> ; PATCH <code>status/tx_hash/block</code>	Low
H3	Auth on tx history by session	Add <code>authMiddleware</code> + owner check to <code>GET /api/transaction/history/:sessionId</code>	Trivial
H4	Extend <code>.env.example</code>	UGF, RPC, contract, Gemini optional vars	Trivial
H5	Frontend UGF execute wiring	Call <code>executeUgfTransaction</code> after user confirms timeline (if not already)	Medium

Medium — polish

ID	Task	Detail	Effort
M1	Wallet balance sync	POST <code>/api/wallet/sync</code> via viem <code>getBalance</code> + UGF vault read	Medium
M2	User counters	Increment <code>total_transactions</code> , <code>total_nfts</code> on confirmed tx	Low
M3	Analytics table	Write events; optional dashboard endpoint	Low
M4	Nonce store persistence	Redis or Supabase for multi-instance / restart safety	Medium

Low — hardening

ID	Task	Detail
L1	Zod schemas for all POST bodies and query params	
L2	Remove dead <code>intentParser.ts</code> , align <code>types/index.ts</code>	
L3	Stricter <code>validateConfig()</code> — fail fast in production without secrets	
L4	OpenAI provider option (optional; Gemini sufficient)	

Database Schema

Reference: `supabase/schema.sql` (if present in repo)

Table	Purpose	Backend usage
users	Wallet identity, mockusd/eth balances, counters	Auth, wallet API
chat_sessions	Per-user threads, title , timestamps	Chat + sessions API
chat_messages	user / assistant messages	Chat + history
transactions	Action log, UGF fields, status	Chat (pending), UGF (confirmed)
minted_badges	NFT metadata, tx_hash	Chat + gallery
ai_actions	Parser audit (intent, confidence)	Chat
analytics	Aggregates	Not used

transactions column population

Column	Chat (POST /chat)	UGF (POST /ugf/execute)
action_type	✓ From intent	✓ From intent
status	✓ pending	✓ success/failed/pending
tx_hash	✗ null	✓ when confirmed
ugf_quote_id	✗	✓ quote digest
gas_fee_mockusd	✓ mock estimate	✓ from UGF quote
block_number , confirmed_at	✗	✓ on success

Environment Variables

Documented in Backend/.env.example

Variable	Required	Used for
PORT	No	Server port (default 5000)
NODE_ENV	No	Environment
GEMINI_API_KEY	Yes*	AI fallback
SUPABASE_URL / keys	Yes*	Database
JWT_SECRET	Yes (prod)	Wallet JWT
FRONTEND_URL	No	CORS

*Warn-only at startup if missing.

Required for UGF execute (add to `.env`)

Variable	Purpose
<code>UGF_API_KEY</code>	UGF SDK authentication
<code>UGF_SIGNER_PRIVATE_KEY</code>	Must match <code>userWallet</code> passed to execute
<code>BASE_SEPOLIA_RPC_URL</code>	Default <code>https://sepolia.base.org</code>
<code>NFT_CONTRACT_ADDRESS</code>	Target contract for calldata

Optional

Variable	Purpose
<code>GEMINI_MODEL</code>	Override default model chain
<code>GEMINI_MAX_RETRIES</code>	Retry count (default 2)
<code>GEMINI_RETRY_DELAY_MS</code>	Backoff (default 600ms)

Frontend ↔ Backend Integration

Capability	Backend	Frontend (<code>src/</code>)	Wired
Wallet auth	✓	<code>WalletAuthSync</code> , <code>authStorage</code>	✓
<code>POST /api/chat</code>	✓	<code>api.ts</code> + Bearer	✓
Chat sessions sidebar	✓	<code>fetchChatSessions</code> , <code>PreviousChats</code>	✓
Load session history	✓	<code>loadSessionHistory</code> , <code>loadSession</code>	✓
Delete session	✓	<code>deleteChatSession</code>	✓
Transaction history panel	✓	<code>GET /api/transactions/:wallet</code>	✓
<code>aiSteps</code> / <code>transactionSteps</code>	✓	<code>useStore</code> simulation	✓
<code>POST /api/ugf/execute</code>	✓	<code>executeUgfTransaction</code> in <code>api.ts</code>	⚠ Verify UX trigger
Gallery API	✓	Partial / mock NFTs in wallet	⚠
Wallet balance sync	✗	Shows DB <code>eth_balance</code>	✗

Judge Demo Checklist

Step	Demo action	Backend ready?	Notes
1	Connect wallet	✓	JWT auth
2	See ETH / Mock USD	⚠	DB values unless manually set
3	Type mint command	✓	Authenticated chat
4	See AI + timeline	✓	Steps in response
5	Previous chats sidebar	✓	Sessions API
6	UGF pays gas (Mock USD)	⚠	Needs C1–C5
7	NFT on-chain + gallery	⚠	DB badge on chat; chain needs UGF

Minimum demo (chat-only): Auth + chat + sessions — **ready now**.

Full Web3 demo: Requires contract + UGF env + execute path tested once.

File Inventory

Active `src/`

File	Status	Role
<code>server.ts</code>	✓	Mounts health, auth, chat, transaction, ugf
<code>routes/health.ts</code>	✓	Health check
<code>routes/auth.ts</code>	✓	Nonce + verify
<code>routes/chat.ts</code>	✓	Chat, sessions, history, delete
<code>routes/transaction.ts</code>	✓	Tx, gallery, wallet
<code>routes/ugf.ts</code>	✓	UGF execute
<code>middleware/authMiddleware.ts</code>	✓	JWT + wallet access
<code>middleware/cors.ts</code>	✓	CORS
<code>middleware/errorHandler.ts</code>	✓	Errors
<code>services/responseEngine.ts</code>	✓	Timeline steps
<code>services/ugfService.ts</code>	✓	UGF SDK flow
<code>services/userService.ts</code>	✓	User CRUD
<code>services/nonceStore.ts</code>	✓	In-memory nonces
<code>services/intentParser.ts</code>	📦	Legacy, unused
<code>config/env.ts</code>	✓	Config
<code>config/supabase.ts</code>	✓	Admin client

Quick Start

```
cd Backend
cp .env.example .env
# Set: GEMINI_API_KEY, SUPABASE_*, JWT_SECRET
# For UGF: UGF_API_KEY, UGF_SIGNER_PRIVATE_KEY, NFT_CONTRACT_ADDRESS

npm install
npm run dev
# → http://localhost:5000
```

Health

```
curl -s http://localhost:5000/health | jq
```

Auth + chat (typical frontend flow)

1. POST /api/auth/nonce with { "walletAddress": "0x..." }
2. Sign returned nonce in wallet
3. POST /api/auth/verify with { "walletAddress", "signature" }
4. POST /api/chat with header Authorization: Bearer <token> and body { "message": "Mint badge for Jay", "sessionId": "<optional-uuid>" }

```
# Example (replace TOKEN):
curl -s -X POST http://localhost:5000/api/chat \
-H "Content-Type: application/json" \
-H "Authorization: Bearer TOKEN" \
-d '{"message": "Mint badge for Jay"}' | jq
```

List sessions

```
curl -s "http://localhost:5000/api/chat/sessions?walletAddress=0x..." \
-H "Authorization: Bearer TOKEN" | jq
```

UGF AgentX Backend · Implementation status

Reflects Backend/src/ as of May 19, 2026